
REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

This lab taught us how to do communications between microprocessors and other devices! Pretty important to know how to send data to and from an MCU to a peripheral and vice versa, as a microprocessor with no data is not useful at all. Could be used to read data from sensors, send controls to peripheral devices, and more, maybe a lighting controller 😊.

PRE-LAB EXERCISES

- i. The sampling rate of a UART receiver is usually faster than the baud rate of the overall system. Why is this so?

Because the processor oversamples to detect when the start bit occurs, and to obtain proper sampling of the incoming bits. UART is Asynchronous which means there is not a shared clock; so oversampling helps to sample each bit at the most-accurate time.

- ii. What is the maximum possible baud rate for asynchronous communication within the USART system of the ATxmega128A1U, assuming that the microcontroller has a system clock frequency of 2 MHz and that the USART “double-speed mode” is disabled (i.e., the relevant bit CLK2X is set to 0)? In addition to the maximum rate, provide the values of the relevant registers used to configure that rate. Whenever appropriate, support your answer with calculations.

We would use Table 23-1, which shows for a BSCALE ≥ 0 we need to use:

$$f_{BAUD} = \frac{f_{PER}}{2^{BSCALE} * 16(BSEL + 1)}$$

BSCALE CAN be -7 to 7 (but a constraint of the equation above is BSCALE ≥ 0) and BSEL can be any value from 0 to 4095; the smaller we make the denominator the higher the baud frequency; so we make BSCALE = 0, and we make BSEL = 0, leaving:

$$f_{BAUD} = \frac{2000000}{16} = 125000 \text{ bits per second}$$

- iii. In the context of the USART system within the ATxmega128A1U, how many buffers (i.e., memory locations that store temporary data) are used by a transmitter? How many are used by a receiver? Additionally, for both transmitters and receivers, explain how the use of buffers provides greater flexibility to an application involving these components.

The transmitter uses a single write buffer (DATA), which lets it have continuous data transmission without any delay by cycling out the current data for the next; the receiver uses a ‘two-level’ receive buffer (DATA) to ensure that the CPU has time to read the data. This ensures that the CPU has more time to read the data and prevent missing any communications.

- iv. If an asynchronous serial communication protocol of 7 data bits, one start bit, one stop bits, odd parity, and baud rate of 37.44 kHz was chosen, calculate how many seconds it would take to transmit the ASCII character string “Dr. Schwartz saw seven slick slimy snakes slowly sliding southward.” (This string has 67 characters.) Note that ASCII is a 7-bit (not an 8-bit) code. Show all work.

37.44kHz = 37440 bits per second; if it takes 7 bits of data, 1 bit for start, 1 bit for stop, and one bit for parity for one full character read, we simply do:

$$(10 \text{ bits} * 67 \text{ characters}) / 37740 \text{ bits per second} = 17.753 \text{ms}$$

PSEUDOCODE/FLOWCHARTS

Lab5_2:

MAIN:

Initialize SP

Rcall USART_INIT

LOOP:

Load R17 with 'U'

Rcall OUT_CHAR

rjmp LOOP

; SUBROUTINES

USART_INIT:

Push r16

; Use PortD

Set the TxD pin high (PORTx_OUT)

Set the TxD pin as output

Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps

; for an awkward baud rate like this, we should use negative BSCALE, and go to the smallest increment possible, which is 2^{-7} . If we do the math we find that:

$$\frac{2000000}{37440} = 16((2^{B_{SCALE}} * B_{SEL}) + 1)$$

$$\frac{53.4188}{16} = 2^{B_{SCALE}} * B_{SEL} + 1$$

$$2.338675 = 2^{B_{SCALE}} * B_{SEL}$$

$$\frac{2.338675}{2^{-7}} = 299 = B_{SEL}, \text{ which matches the Excel Calculator}$$

Set the frame format / mode in CTRLC

Enable the transmit/receive in CTRLB

Set up Interrupts in CTRLA

Pop r16

ret

OUT_CHAR:

Push r16

; We will first check if there is a transmission in progress, by checking the TXCIF flag in the STATUS register. (bit 6)

WAIT:

Load TXCIF flag in STATUS register, check bit 6 is 1, else branch to WAIT

Load USARTD0_DATA with R17

Pop r16

Ret

Lab5_3:

Same as Lab5_2 but using PORTC instead of PORTD

Lab5_4:

Include prior OUT_CHAR, USART_INIT subroutines from Lab5_2

.org 0

Rjmp MAIN

.org 0x2000

STRING_ADDR:

; input table data here for string

MAIN:

Initialize Stack Pointer

Rcall USART_INIT

Set Z to point to STRING_ADDR

LOOP:

Rjmp LOOP

; --SUBROUTINES-- ;

OUT_STRING:

Push r16

Lpm r17, Z+

Test r17 for zero flag

Branch if not equal to SKIP

If it IS zero we will reset pointers

SKIP: ; No zero flag so we continue writing the string

Rcall OUT_CHAR

Pop r16

Ret

Lab5_5:

; Include the subroutines: OUT_CHAR, USART_INIT (for PORTD) and the associated code.

MAIN:

Set up SP

Call USART_INIT

LOOP:

Call IN_CHAR

Rjmp LOOP

; SUBROUTINES:

; Use subroutines mentioned above from Lab5_4

IN_CHAR:

Push r16

WAIT_IN:

Load status flags

Check if the receive flag is set

Rjmp to WAIT_IN if not

Load r17 with USARTD0_DATA

Rcall OUT_CHAR

Pop r16

Ret

Lab5_6:

; Move over subroutines from Lab5_5, with modified IN_CHAR shown below and new IN_STRING

.equ STR_SIZE = 64

.org 0

Rjmp MAIN

.dseg

.org 0x1000

STR_STORAGE: .byte STR_SIZE

.org 0x100

MAIN:

Initialize SP

Call USART_INIT

Initialize Pointer Y to dseg 0x1000

LOOP:

Call IN_STRING

Rjmp LOOP

; SUBROUTINES

IN_STRING:

Push r16

LOOP_INSTR:

Rcall IN_CHAR

Check for Carriage Return, if true branch to DONE_STRING

Check for Backspace, if true branch to BACK_SPACE

Check for Delete, if true branch to BACK_SPACE

Store r17 to y, post-increment

Rcall OUT_CHAR

Rjmp LOOP_INSTR ; continue loop, ignore conditional statements below

BACK_SPACE:

Sbiw Y, 1

Rjmp LOOP_INSTR

DONE_STRING:

Ldi r17, 0x00

St Y, r17

Pop r16

ret

OUT_STRING:

Push r16

LOOP_OUT:

; Set Y to beginning of string memory

Ld r17, Y+

Tst r17

Branch to DONE_OUT if null

Rcall OUT_CHAR

Rjmp LOOP_OUT

DONE_OUT:

Pop r16

ret

Lab5_7:

; Use USART initialization from previous sections, add interrupt initialization in CTRLA; add a TC

.org 0

Rjmp MAIN

.org USART_INT_VECTOR

Rjmp INTERRUPT

MAIN:

Set PD6 (BLUE_PWM) as an output

Initialize SP

rcall USART_INIT

rcall USART_INTERRUPT_INIT

sei

LOOP:

Toggle PD6 Output

Rjmp LOOP

; SUBROUTINES & INTERRUPTS

PMIC_INIT:

Push r16

Enable the PMIC for low level interrupts

Pop r16

Ret

INTERRUPT:

Push r16

Lds r16, CPU_SREG

Push r16

Rcall IN_CHAR

Rcall OUT_CHAR

Pop r16

Sts CPU_SREG, r16

Pop r16

reti

PROGRAM CODE

```
Lab5_2:
;
; Lab5.asm
;
; Created: 7/4/2026 3:48:55 PM
; Author : Evan Baesler
;

.include "ATXmega128A1Udef.inc"

.equ B_SEL = 299
.equ B_SCALE = -7

.org 0
rjmp MAIN

.org 0x100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT

LOOP:
ldi r17, 'U'
rcall OUT_CHAR
rjmp LOOP
```

```
; --SUBROUTINES-- ;
USART_INIT:
push r16

; Set the TxD pin high (PORTx_OUT) (PORTC3)
ldi r16, (1<<3)
sts PORTD_OUT, r16

; Set the TxD pin as output
sts PORTD_DIRSET, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTD0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTD0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC

; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTD0_CTRLC, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTD0_CTRLB
ori r16, 0b00001000 ; Enable Rx/Tx without changing previous defaults
sts USARTD0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)

pop r16
ret

OUT_CHAR:
push r16

WAIT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrc r16, USART_DREIF_bp
rjmp WAIT

sts USARTD0_DATA, r17 ; Data input from main loop

pop r16
ret
```

Lab5_3:

```
/*
 * Testing.asm
 *
 * Created: 7/4/2026 6:47:03 PM
 * Author: Evan Baesler
 */

.include "ATXmega128A1Udef.inc"

.equ B_SEL = 299
.equ B_SCALE = -7

.org 0
rjmp MAIN

.org 0x100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT

LOOP:
ldi r17, 'U'
rcall OUT_CHAR
rjmp LOOP
```

```
; --SUBROUTINES-- ;
USART_INIT:
push r16

; Set the TxC pin high (PORTx_OUT) (PORTC3)
ldi r16, (1<<3)
sts PORTC_OUT, r16

; Set the TxD pin as output
sts PORTC_DIRSET, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTC0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTC0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC
; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTC0_CTRLC, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTC0_CTRLB
ori r16, 0b00001000 ; Enable Rx/Tx without changing previous defaults
sts USARTC0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)

pop r16
ret

OUT_CHAR:
push r16

WAIT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTC0_STATUS
sbrs r16, USART_DREIF_bp
rjmp WAIT

sts USARTC0_DATA, r17 ; Data input from main loop

pop r16
ret
```

Lab5_4:

```
/*
 * Lab5_4.asm
 *
 * Created: 7/4/2026 9:00:37 PM
 * Author: Evan Baesler
 */

.equ B_SCALE = -7
.equ B_SEL = 299

.org 0
rjmp MAIN

.org 0x2000
STRING_ADDR: ; String Data
.db 'H', 'i', ' ', 'H', 'o', 'w', ' ', 'A', 'r', 'e', ' ', 'Y', 'o', 'u', '?', ' ', ' ', 0

.org 0x100
MAIN:
ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT

ldi ZL, low(STRING_ADDR << 1)
ldi ZH, high(STRING_ADDR << 1)

LOOP:

rcall OUT_STRING

rjmp LOOP
```

```
; --SUBROUTINES-- ;
OUT_STRING:
push r16
lpm r17, Z+
; branch if null operator
tst r17
brne SKIP

ldi ZL, low(String_ADDR << 1)
ldi ZH, high(String_ADDR << 1)

SKIP:
rcall OUT_CHAR
pop r16
ret

OUT_CHAR:
push r16

WAIT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_DREIF_bp
rjmp WAIT

sts USARTD0_DATA, r17 ; Data input from main loop

pop r16
ret
```

```
USART_INIT:
push r16

; Set the TxD pin high (PORTx_OUT) (PORTD3)
ldi r16, (1<<3)
sts PORTD_OUT, r16

; Set the TxD pin as output
sts PORTD_DIRSET, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTD0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTD0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC
; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTD0_CTRLC, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTD0_CTRLB
ori r16, 0b00001000 ; Enable Rx/Tx without changing previous defaults
sts USARTD0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)

pop r16
ret
```

Lab5_5:

```
/*
 * Lab5_5.asm
 *
 * Created: 7/4/2026 9:32:48 PM
 * Author: Evan Baesler
 */

; Echo program

.equ B_SCALE = -7
.equ B_SEL = 299

.org 0
rjmp MAIN

.org 0x100
MAIN:
ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT

LOOP:

rcall IN_CHAR

rjmp LOOP

; --SUBROUTINES-- ;
IN_CHAR:
push r16
WAIT_IN:
; We check the STATUS register for RXCIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_RXCIF_bp
rjmp WAIT_IN

lds r17, USARTD0_DATA
rcall OUT_CHAR
pop r16
ret

OUT_CHAR:
push r16

WAIT_OUT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_DREIF_bp
rjmp WAIT_OUT

sts USARTD0_DATA, r17 ; Data input from main loop

pop r16
ret
```

```
USART_INIT:
push r16

; Set the TxD/RxD pin high (PORTx_OUT) (PORTD[4:3])
ldi r16, 0b00011000
sts PORTD_OUT, r16

; Set the TxD pin as output
ldi r16, (1<<3)
sts PORTD_DIRSET, r16
ldi r16, (1<<4)
sts PORTD_DIRCLR, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTD0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTD0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC
; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTD0_CTRLC, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTD0_CTRLB
ori r16, 0b00011000 ; Enable Rx/Tx without changing previous defaults
sts USARTD0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)

pop r16
ret
```

Lab5_6:

```
/*
 * Lab5_6.asm
 *
 * Created: 7/5/2026 4:00:29 PM
 * Author: Evan Baesler
 */

; Move over subroutines from Lab5_5, with modified IN_CHAR shown below and new IN_STRING
.equ STR_SIZE = 0x100
.equ B_SEL = 299
.equ B_SCALE = -7

.org 0
Rjmp MAIN

.dseg
.org 0x2000
STR_STORAGE: .byte STR_SIZE

.cseg
.org 0x100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT

LOOP:

; Reset Y to start of buffer before input
ldi YL, low(STR_STORAGE)
ldi YH, high(STR_STORAGE)
rcall IN_STRING

; Now echo the string
ldi YL, low(STR_STORAGE)
ldi YH, high(STR_STORAGE)
rcall OUT_STRING

; SUBROUTINES
IN_STRING:
push r16
push YL
push YH

LOOP_INSTR:
rcall IN_CHAR ; loads r17 with input character

; Check for Carriage Return, if true branch to DONE_STRING
cpi r17, 0x0D
breq DONE_STRING

; Check for Backspace, if true branch to BACK_SPACE
cpi r17, 0x08
breq BACK_SPACE
cpi r17, 0x7F
```

```
breq BACK_SPACE

; Store r17 to y, post-increment
st Y+, r17
rcall OUT_CHAR
rjmp LOOP_INSTR ; continue loop, ignore conditional statements below

BACK_SPACE:
cpi YL, low(STR_STORAGE)
brne BACK_SPACE_OK
cpi YH, high(STR_STORAGE)
breq LOOP_INSTR

BACK_SPACE_OK:
sbiw Y, 1 ; Push pointer back & replace with ' '
ldi r17, 0x08
rcall OUT_CHAR
ldi r17, ' '
rcall OUT_CHAR
ldi r17, 0x08
rcall OUT_CHAR

rjmp LOOP_INSTR

DONE_STRING:
ldi r17, 0x00
st Y, r17

pop YH
pop YL
pop r16
ret

; -----

OUT_STRING:
push r16

ldi r17, 0x0D
rcall OUT_CHAR

ldi YL, low(STR_STORAGE)
ldi YH, high(STR_STORAGE)

LOOP_OUT:
ld r17, Y+
tst r17
breq DONE_OUT

rcall OUT_CHAR
rjmp LOOP_OUT

DONE_OUT:
ldi r17, 0x0D
rcall OUT_CHAR
pop r16
ret

; -----

IN_CHAR:
```

```
push r16
WAIT_IN:
; We check the STATUS register for RXCIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_RXCIF_bp
rjmp WAIT_IN

lds r17, USARTD0_DATA
pop r16
ret

; -----

OUT_CHAR:
push r16

WAIT_OUT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_DREIF_bp
rjmp WAIT_OUT

sts USARTD0_DATA, r17 ; Data input from main loop
pop r16
ret

; -----

USART_INIT:
push r16

; Set the TxD/RxD pin high (PORTx_OUT) (PORTD[4:3])
ldi r16, 0b00011000
sts PORTD_OUT, r16

; Set the TxD pin as output
ldi r16, (1<<3)
sts PORTD_DIRSET, r16
ldi r16, (1<<4)
sts PORTD_DIRCLR, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTD0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTD0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC
; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
```

```
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCHRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTD0_CTRLA, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTD0_CTRLB
ori r16, 0b00011000 ; Enable Rx/Tx without changing previous defaults
sts USARTD0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)

pop r16
ret
```

Lab5_7:

```
/*
 * Lab5_7.asm
 *
 * Created: 7/5/2026 7:44:43 PM
 * Author: Evan Baesler
 */

.equ B_SEL = 299
.equ B_SCALE = -7

; Use USART initialization from previous sections
.org 0
rjmp MAIN

.org USARTD0_RXC_vect
rjmp INTERRUPT

MAIN:
ldi r16, (1<<6) ; PD6 (BLUE_PWM) as an output
sts PORTD_DIRSET, r16

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

rcall USART_INIT
rcall TC_INIT
rcall PMIC_INIT

sei
LOOP:

lds r16, TCC0_INTFLAGS ; Read the polling timer interrupt flags
sbrs r16, 0 ; Skip the toggle if the OVF (bit 0) is NOT set
rjmp LOOP ; If flag is 0, keep looping

ldi r16, (1<<6)
sts PORTD_OUTTGL, r16

ldi r16, 1 ; clear OVIF by writing 1 to it
sts TCC0_INTFLAGS, r16
rjmp LOOP

; SUBROUTINES & INTERRUPTS
PMIC_INIT:
push r16
ldi r16, PMIC_LOLVLEN_bm
sts PMIC_CTRL, r16
Pop r16
Ret

USART_INIT:
push r16

; Set the TxD/RxD pin high (PORTx_OUT) (PORTD[4:3])
ldi r16, 0b00011000
```

```
sts PORTD_OUT, r16

; Set the TxD pin as output
ldi r16, (1<<3)
sts PORTD_DIRSET, r16
ldi r16, (1<<4)
sts PORTD_DIRCLR, r16

; Select BSCALE and BSEL in BAUDCTRLA/B for 37,440 bps
; Math Shown on PDF in Pseudocode, derived for BSCALE = -7, BSEL = 299
; BAUDCTRLA = BSEL[7:0]
; BAUDCTRLB = BSCALE[3:0] : BSEL[11:8]
; BSCALE = 1001 = -7 in 2's Comp
; BSEL = 256 + 32 + 8 + 2 + 1 = 2^8 + 2^5 + 2^3 + 2^1 + 2^0
; BAUDCTRLA = 00101011
; BAUDCTRLB = 10010001
ldi r16, low(B_SEL)
sts USARTD0_BAUDCTRLA, r16
ldi r16, ((B_SCALE << 4) | high(B_SEL))
sts USARTD0_BAUDCTRLB, r16

; Set the frame format / mode in CTRLC
; In NON-SPI:
; CTRLC = CMODE[1:0] : PMODE[1:0] : SBMODE : CHSIZE[2:0]
; CMODE = 00 for Asynchronous
; PMODE = 10 for Even Parity
; SBMODE = 0 for 1 Stop Bit
; CHSIZE = 011 for 8-bit data
ldi r16, (USART_CMODE_ASYNCRONOUS_gc | USART_PMODE_EVEN_gc | USART_CHSIZE_8BIT_gc)
sts USARTD0_CTRLC, r16

; Enable the transmit/receive in CTRLB (CTRLB[4:3]) 4 = RXEN, 3 = TXEN
lds r16, USARTD0_CTRLB
ori r16, 0b00011000 ; Enable Rx/Tx without changing previous defaults
sts USARTD0_CTRLB, r16

; Set up Interrupts in CTRLA (NOT UTILIZED HERE)
ldi r16, USART_RXCINTLVL_LO_gc
sts USARTD0_CTRLA, r16

pop r16
ret

OUT_CHAR:
push r16

WAIT_OUT:
; We check the STATUS register for DREIF, which tells us the current
; transmission is done when the flag goes high. (bit 5)
lds r16, USARTD0_STATUS
sbrs r16, USART_DREIF_bp
rjmp WAIT_OUT

sts USARTD0_DATA, r17 ; Data input from main loop
pop r16
ret
```

```
TC_INIT:
push r16
; Set the Timer Period to 31,250 cycles for a prescaler of 64, for 1s period
ldi r16, low(31250)
sts TCC0_PER, r16
ldi r16, high(31250)
sts TCC0_PER+1, r16
ldi r16, TC_CLKSEL_DIV64_gc
sts TCC0_CTRLA, r16
pop r16
ret
```

```
INTERRUPT:
push r16
lds r16, CPU_SREG
push r16

lds r17, USARTD0_DATA ; read directly rather than waiting
                        ; to prevent holding up loop

rcall OUT_CHAR

pop r16
sts CPU_SREG, r16
pop r16
reti
```

APPENDIX

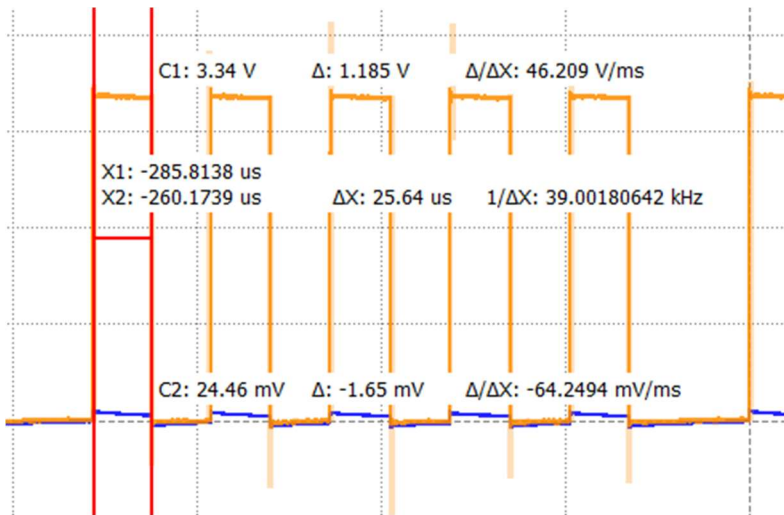


Figure 1: Bit Transmission Measurement (Lab5_3)

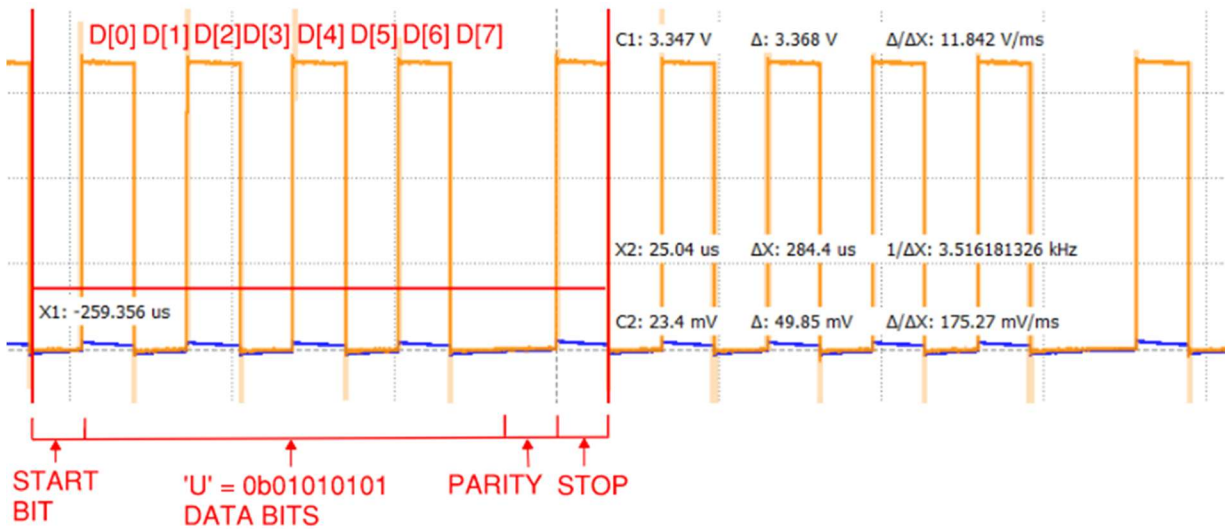


Figure 2: Character Transmission Measurement (Lab5_3)

Word Address		
ATxmega128A1U		ATxmega64A1U
0		0
FFFF	/	77FF
F000	/	7800
FFFF	/	7FFF
10000	/	8000
10FFF	/	87FF

Application Section (bytes)
(128K/64K)

...

Application Table Section (bytes) (8K/
4K)

Boot Section (bytes)
(8K/4K)

Figure 3: Program Memory Addressing

Byte Address	ATxmega64A1U
0	I/O Registers (4K)
FFF	
1000	EEPROM (2K)
17FF	
	RESERVED
2000	Internal SRAM (4K)
2FFF	
4000	External Memory (0 to 16MB)
FFFFFF	

Byte Address	ATxmega128A1U
0	I/O Registers (4KB)
FFF	
1000	EEPROM (2K)
17FF	
	RESERVED
2000	Internal SRAM (8K)
3FFF	
3000	External Memory (0 to 16MB)
FFFFFF	

Figure 4: Data Memory Addressing

13.16 Register summary – Ports

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DIR	DIR[7:0]								148
+0x01	DIRSET	DIRSET[7:0]								148
+0x02	DIRCLR	DIRCLR[7:0]								148
+0x03	DIRTGL	DIRTGL[7:0]								148
+0x04	OUT	OUT[7:0]								149
+0x05	OUTSET	OUTSET[7:0]								149
+0x06	OUTCLR	OUTCLR[7:0]								149
+0x07	OUTTGL	OUTTGL[7:0]								149
+0x08	IN	IN[7:0]								150
+0x09	INTCTRL	–	–	–	–	INT1LVL[1:0]		INT0LVL[1:0]		150
+0x0A	INT0MASK	INT0MSK[7:0]								150
+0x0B	INT1MASK	INT1MSK[7:0]								150
+0x0C	INTFLAGS	–	–	–	–	–	–	INT1IF	INT0IF	151
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	REMAP	–	–	SPI	USART0	TC0D	TC0C	TC0B	TC0A	151
+0x0F	Reserved	–	–	–	–	–	–	–	–	
+0x10	PIN0CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x11	PIN1CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x12	PIN2CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x13	PIN3CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x14	PIN4CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x15	PIN5CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x16	PIN6CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x17	PIN7CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x18	Reserved	–	–	–	–	–	–	–	–	
+0x19	Reserved	–	–	–	–	–	–	–	–	
+0x1A	Reserved	–	–	–	–	–	–	–	–	
+0x1B	Reserved	–	–	–	–	–	–	–	–	
+0x1C	Reserved	–	–	–	–	–	–	–	–	
+0x1D	Reserved	–	–	–	–	–	–	–	–	
+0x1E	Reserved	–	–	–	–	–	–	–	–	
+0x1F	Reserved	–	–	–	–	–	–	–	–	

Figure 5: Ports Register Summary

14.13 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA	–	–	–	–	CLKSEL[3:0]				175
+0x01	CTRLB	CCDEN	CCCEN	CCBEN	CCAEN	–	WGMode[2:0]			175
+0x02	CTRLC	–	–	–	–	CMPD	CMPC	CMPB	CMPA	176
+0x03	CTRLD	EVACT[2:0]			EVDLY	EVSEL[3:0]				176
+0x04	CTRLF	–	–	–	–	–	–	BYTEM		178
+0x05	Reserved	–	–	–	–	–	–	–	–	
+0x06	INTCTRLA	–	–	–	–	ERRINTLVL[1:0]		OVINTLVL[1:0]		178
+0x07	INTCTRLB	CCCINTLVL[1:0]		CCBINTLVL[1:0]		CCAINTLVL[1:0]		CCBINTLVL[1:0]		178
+0x08	CTRLCLR	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	179
+0x09	CTRLFSET	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	180
+0x0A	CTRLGCLR	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0B	CTRLGSET	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0C	INTFLAGS	CCDIF	CCCIF	CCBIF	CCAIF	–	–	ERRIF	OVFIF	180
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	TEMP	TEMP[7:0]								181
+0x10 to +0x1F	Reserved	–	–	–	–	–	–	–	–	
+0x20	CNTL	CNT[7:0]								181
+0x21	CNTH	CNT[15:8]								181
+0x22 to +0x25	Reserved	–	–	–	–	–	–	–	–	
+0x26	PERL	PER[7:0]								182
+0x27	PERH	PER[8:15]								182
+0x28	CCAL	CCA[7:0]								182
+0x29	CAAH	CCA[15:8]								182
+0x2A	CCBL	CCB[7:0]								182
+0x2B	CBH	CCB[15:8]								182
+0x2C	CCCL	CCC[7:0]								182
+0x2D	CCCH	CCC[15:8]								182
+0x2E	CCDL	CCD[7:0]								182
+0x2F	CCDH	CCD[15:8]								182
+0x30 to +0x35	Reserved	–	–	–	–	–	–	–	–	
+0x36	PERBUFL	PERBUF[7:0]								183

Figure 6: Timer/Counter Register Summary

12.9 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	STATUS	NMIEX	–	–	–	–	HILVLEX	MEDLVLEX	LOLVLEX	136
+0x01	INTPRI	INTPRI[7:0]								136
+0x02	CTRL	RREN	IVSEL	–	–	–	HILVLEN	MEDLVLEN	LOLVLEN	137

Figure 7: Interrupts Register Summary

27.12 Register summary – EBI

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	SDDATAW[1:0]		LPCMODE[1:0]		SRMODE[1:0]		IFMODE[1:0]		329
+0x01	SDRAMCTRLA	—	—	—	—	SDCAS	SDROW	SDCOL[1:0]		330
+0x02	Reserved	—	—	—	—	—	—	—	—	
+0x03	Reserved	—	—	—	—	—	—	—	—	
+0x04	REFRESHL	SDRAM Refresh Period Low Byte								331
+0x05	REFRESHH	—	—	—	—	—	—	SDRAM Refresh Period High		331
+0x06	INITDLYL	SDRAM Initialization Time Low Byte								331
+0x07	INITDLYH	—	—	SDRAM Initialization Time High Byte						331
+0x08	SDRAMCTRLB	MRDLY[1:0]		ROWCYCDLY[[2:0]			RPDLY[2:0]			332
+0x09	SDRAMCTRLC	WRDLY[1:0]		ESRDLY[2:0]			ROWCOLDLY[2:0]			333
+0x0A	Reserved	—	—	—	—	—	—	—	—	
+0x0B	Reserved	—	—	—	—	—	—	—	—	
+0x0C	Reserved	—	—	—	—	—	—	—	—	
+0x0D	Reserved	—	—	—	—	—	—	—	—	
+0x0E	Reserved	—	—	—	—	—	—	—	—	
+0x0F	Reserved	—	—	—	—	—	—	—	—	
+0x10	CS0	Chip Select 0 Offset Address								
+0x14	CS1	Chip Select 1 Offset Address								
+0x18	CS2	Chip Select 2 Offset Address								
+0x1C	CS3	Chip Select 3 Offset Address								

Figure 8: EBI Register Summary

27.13 Register summary – EBI chip select

Address	Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA		–	ASIZE[4:0]					MODE[1:0]		329
+0x01	CTRLB	(SRAM)	–	–	–	–	–	SRWS[2:0]			330
		(SDRAM)	SDINITDONE	–	–	–	–	SDSREN	SDMODE[1:0]		
+0x02	BASEADDRL		Chip Select Base Address Low Byte				–	–	–	–	337
+0x03	BASEADDRH		Chip Select Base Address High Byte								337

Figure 9: EBI Chip Select Register Summary

Table 33-7. Port H - alternate functions.

PORT H	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	53							
VCC	54							
PH0	55	SYNC	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$
PH1	56	SYNC	$\overline{CAS}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$
PH2	57	SYNC/ASYN	$\overline{RAS}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$
PH3	58	SYNC	$\overline{DQM}$		$\overline{ALE2}$			$\overline{ALE2}$
PH4	59	SYNC	BA0	$\overline{CS0/A16}$	$\overline{CS0}$	$\overline{CS0/A16}$	$\overline{CS0}$	$\overline{CS0/A16}$
PH5	60	SYNC	BA1	$\overline{CS1/A17}$	$\overline{CS1}$	$\overline{CS1/A17}$	$\overline{CS1}$	$\overline{CS1/A17}$
PH6	61	SYNC	CKE	$\overline{CS2/A18}$	$\overline{CS2}$	$\overline{CS2/A18}$	$\overline{CS2}$	$\overline{CS2/A18}$
PH7	62	SYNC	CLK	$\overline{CS3/A19}$	$\overline{CS3}$	$\overline{CS3/A19}$	$\overline{CS3}$	$\overline{CS3/A19}$

Notes: 1. CS0 - CS3 can optionally be moved to Port E or F
2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 10: PORTH Alternate Functions

Table 33-8. Port J - alternate functions.

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	63							
VCC	64							
PJ0	65	SYNC	D0	D0	D0	D0/A0	D0/A0	D0/A0/A8
PJ1	66	SYNC	D1	D1	D1	D1/A1	D1/A1	D1/A1/A9

ATxmega128A1U/ATxmega64A1U

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
PJ2	67	SYNC/ASYN	D2	D2	D2	D2/A2	D2/A2	D2/A2/A10
PJ3	68	SYNC	D3	D3	D3	D3/A3	D3/A3	D3/A3/A11
PJ4	69	SYNC	A8	D4	D4	D4/A4	D4/A4	D4/A4/A12
PJ5	70	SYNC	A9	D5	D5	D5/A5	D5/A5	D5/A5/A13
PJ6	71	SYNC	A10	D6	D6	D6/A6	D6/A6	D6/A6/A14
PJ7	72	SYNC	A11	D7	D7	D7/A7	D7/A7	D7/A7/A15

Figure 11: PORTJ Alternate Functions

Table 33-9. Port K - alternate functions.

PORT K	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE2	LPC3 ALE1
GND	73					
VCC	74					
PK0	75	SYNC	A0	A0/A8	A0/A8/A16	A8
PK1	76	SYNC	A1	A1/A9	A1/A9/A17	A9
PK2	77	SYNC/ASYNC	A2	A2/A10	A2/A10/A18	A10
PK3	78	SYNC	A3	A3/A11	A3/A11/A19	A11
PK4	79	SYNC	A4	A4/A12	A4/A12/A20	A12
PK5	80	SYNC	A5	A5/A13	A5/A13/A21	A13
PK6	81	SYNC	A6	A6/A14	A6/A14/A22	A14
PK7	82	SYNC	A7	A7/A15	A7/A15/A23	A15

Figure 12: PORTK Alternate Functions

23.16 Register summary

23.16.1 Register description – USART

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	FERR	BUOVF	PERR	–	RXB8	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	CLK2X	MPCM	TXB8	296
+0x05	CTRLC	CMODE[1:0]		PMODE[1:0]		SBMODE	CHSIZE[2:0]			296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

23.16.2 Register description – USART in SPI Master Mode

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	–	–	–	–	–	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	–	–	–	296
+0x05	CTRLC	CMODE[1:0]		–	–	–	UDORD	UCPHA	–	296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

Figure 13: USART Register Summaries

Table 33-3. Port C - alternate functions.

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USARTC0 ⁽³⁾	USARTC1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
GND	13										
VCC	14										
PC0	15	SYNC	OC0A	OC0ALS					SDA		

ATxmega128A1U/ATxmega64A1U

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USARTC0 ⁽³⁾	USARTC1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
PC1	16	SYNC	OC0B	OC0AHS		XCK0			SCL		
PC2	17	SYNC/ASYNC	OC0C	OC0BLS		RXD0					
PC3	18	SYNC	OC0D	OC0BHS		TXD0					
PC4	19	SYNC		OC0CLS	OC1A			SS			
PC5	20	SYNC		OC0CHS	OC1B		XCK1	MOSI			
PC6	21	SYNC		OC0DLS			RXD1	MISO		clk _{RTC}	
PC7	22	SYNC		OC0DHS			TXD1	SCK		clk _{PER}	EVOUT

- Notes:
1. Pin mapping of all TC0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 2. If TC0 is configured as TC2 all eight pins can be used for PWM output. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 3. Pin mapping of all USART0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 4. Pins MOSI and SCK for all SPI can optionally be swapped. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 5. CLKOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.
 6. EVOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.

Figure 14: PORTC Alternate Functions